

Relatório de Auditoria de Segurança

Ateliê ERP + DeskcommCRM

Cloudflare Workers · Astro 7 · D1 · TypeScript

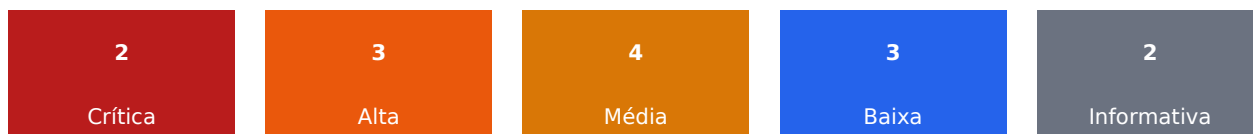
Data da auditoria	02/10/2026
Escopo	src/** (rotas de API, camada de auth/authz, front framework-free, migrations, configs de deploy)
Metodologia	Análise estática arquivo a arquivo + rastreamento de todos os handlers de rota
Achados totais	14 (critica=2, alta=3, media=4, baixa=3, informativa=2)

Nota metodológica — mapeamento das 5 categorias para a stack detectada

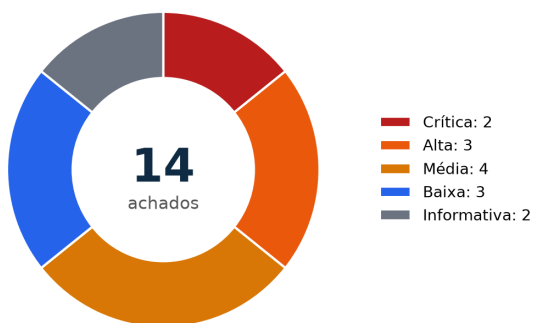
1. Banco sem tranca	Sem RLS (não é Supabase). Verificou-se o isolamento de inquilino/dono nas listagens/agregações do D1: matriz RBAC em authz.ts + columns assignedUserId/assigneeUserId + recorte do SSE. Apontou onde falta recorte por dono.
2. Permissão no navegador	Front framework-free: cruza cada gate de papel da UI (isAdmin/canEdit/role) com o endpoint correspondente e confere se o backend valida o privilégio em toda rota sensível (requireRole/requireRank).
3. IDOR	Percorreu 100% dos handlers PUT/DELETE por ID (fábrica createItemRoutes + rotas dedicadas) e validou a checagem de posse (OWNERSHIP/checkOwnership) em cada um.
4. Chaves expostas	Buscou segredos hardcoded, defaults públicos e ausência de validação de startup em src/, configs, docker-compose/CI; conferiu git (dev.vars gitignored) e o bundle do front por chaves embutidas.
5. Inputs sem tratamento (XSS)	Localizou innerHTML/dangerouslySetInnerHTML/v-html/eval e a aplicação de escapeText/escapeAtrib em cada sink; checou a presença e o uso de sanitização no front e no back.

1. Resumo executivo

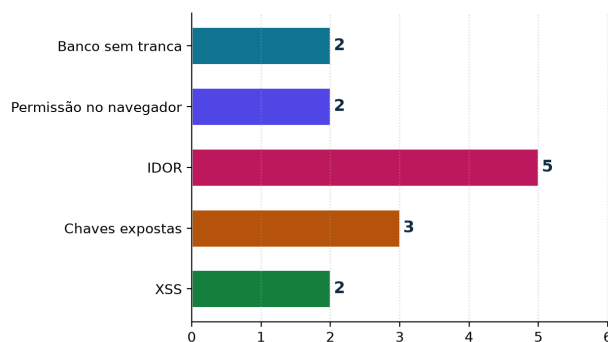
A auditoria identificou **14 achados** distribuídos nas 5 categorias. São **2 críticos**, 3 altos, 4 médios, 3 baixos e 2 informativos. Os riscos centrais concentram-se no IDOR das tabelas do ERP (sem verificação de posse) e na ausência de isolamento de leitura (single-tenant sem recorte por dono). A camada de autenticação, RBAC, auditoria e proteção de origem é sólida.



Achados por severidade



Achados por categoria



2. Pontos fortes e fracos

2.1 Pontos fortes (verificados no código)

ok	Autenticação forte	PBKDF2-SHA256 com 100 mil iterações e salt aleatório por usuário; hash/salt em colunas separadas e falha fechada quando ausente (auth.ts:41-55, 244-258).
ok	Sessões com expiração e rotação	TTL de 30 dias, cookie HttpOnly + Secure + SameSite=Strict, expurgo de sessões expiradas e rotação no login (auth.ts:26, 71-87, 61-62).
ok	RBAC validado no servidor em todas as rotas sensíveis	requireRole/requireRank aplicados em /api/users, /api/settings, /api/whatsapp/session e /api/whatsapp/send; a fábrica de rotas aplica a matriz viewer/agent/manager a todo CRUD (authz.ts:52-73, routeFactory.ts:56-60).
ok	Ownership check no CRUD do CRM	checkOwnership recusa agent/viewer quando a coluna de dono está vazia (fail-closed) e rejeita escrita em registro alheio; mensagens herdaram posse da conversa (routeFactory.ts:91-141).
ok	Campos imutáveis protegidos	IMMUTABLE_FIELDS bloqueia alteração de createdBy/createdAt/assignedUserId (conversations) e dos campos de autoria de mensagem no PUT (routeFactory.ts:168-183).
ok	Validação de entrada no servidor	numericProblems rejeita NaN/Infinity em campos numéricos; refuseEmptyOwner barra assignedUserId=""; settings filtra por allowlist e converte para número (mapping.ts, routeFactory.ts:185-222, settings.ts:32-44).
ok	Auditoria de ações	action_logs e auth_audit registram CRUD, logins, trocas de senha e ações LGPD; dados de data/export/erase são recortados pelo titular (audit.ts, me/data.ts, me/erase.ts).
ok	Proteção de origem nos endpoints de sessão	assertSameOrigin recusa login/logout/troca de senha com Origin de outra origem, barrando CSRF de sessão (origin.ts:10-20).
ok	HMAC do webhook WAHA com fail-closed	Assinatura SHA-512 verificada em tempo constante; segredos placeholder/curtos são rejeitados; sem segredo útil o receiver devolve 503 secret_required (wahaWebhook.ts:80-133).
ok	XSS mitigado na borda do front	escapeText (via textContent) e escapeAtrib aplicados a todo dado dinâmico antes do innerHTML; nenhum dangerouslySetInnerHTML/v-html/eval/new Function localizado (domain/format.ts:33-47).
ok	Superfície enxuta (framework-free)	Sem React/Radix/shadcn; as views são DOM imperativo com escapatório único de render — reduz vetores de XSS do ecossistema (tests/dependencies.test.ts trava a regressão).

2.2 Pontos fracos (riscos centrais)

!	IDOR nas tabelas do ERP	Escrita/deleção de qualquer registro do ERP sem verificação de posse (F6).
!	Gestão de usuários aberta	Manager edita qualquer usuário; escrita cruzada sobre contas (F7).
!	Sem isolamento de leitura	Listagens devolvem a base inteira por papel, sem recorte por dono (F1).
!	Segredo-valor conhecido no source	Placeholder de WAHA commitado e config HMAC que pode operar sem assinatura (F11, F12).

3. Achados detalhados por categoria

Severidade	Arquivo : linha	Descrição
Alta	routeFactory.ts:224-234 (listHandler); todos os GET /api/**/index.ts	F1 · Banco sem tranca Todos os endpoints de listagem (contatos, conversas, mensagens, deals, pedidos, ingredientes, produtos, clientes) devolvem a base inteira, filtrando apenas por papel mínimo (viewer). Não há recorte por assignedUserId/tenant.
Média	events.ts:27-39, 49-103	F2 · Banco sem tranca O recorte de visibilidade do SSE (visibleConversationIds) cobre apenas conversations/messages. Nenhum outro recurso exposto por SSE é filtrado por ownership.
Baixa	CrmContatosView.ts:205-211; crm/conversations/index.ts	F4 · Permissão no navegador A UI só oferece 'Conversar' para contatos com telefone, mas POST /api/crm/conversations valida apenas o papel (agent) e não a viabilidade do canal/contato.
Média	CrmInboxView.ts:492-506; orders/index.ts	F5 · Permissão no navegador O compositor de pedido da inbox vincula o cliente do contato em aberto, mas POST /api/orders aceita qualquer customerId do body. O agente pode criar pedidos para clientes fora da conversa aberta.
Crítica	routeFactory.ts:73-100 (OWNERSHIP + checkOwnership)	F6 · IDOR Tabelas do ERP (ingredients, components, products, customers, orders, stock_movements) e demais (settings, pipelines, stages, tags...) não possuem coluna de dono no mapa OWNERSHIP. O checkOwnership devolve null (permite) para elas.
Crítica	users/[id].ts:28-65 (allowedRoles, PUT/DELETE)	F7 · IDOR PUT /api/users/:id aceita, para manager, a edição de qualquer campo (exceto role/password sem admin) de qualquer usuário. Um manager pode mudar name/email de qualquer um e, como não há checagem de posse entre pares, a gestão de equipe fica aberta a escrita mútua.
Alta	crm/quick-replies/[id].ts, crm/tags/[id].ts, crm/appoi ntment-types/[id].ts, crm/pipelines/[id].ts, crm/stages/[id].ts	F8 · IDOR Tabelas CRM de apoio (quick_replies, tags, appointment_types, pipelines, pipeline_stages) não estão em OWNERSHIP; qualquer agent pode editar/deletar respostas, etiquetas e estrutura do funil de todo o time.
Média	crm/messages/index.ts (POST); routeFactory.ts:248-255	F9 · IDOR Na criação de mensagem, o conversationId vem do body do cliente e não é validado contra a posse da conversa. O checkOwnership só atua no PUT/DELETE (via conversas).

Severidade	Arquivo : linha	Descrição
Baixa	tables.ts:216-224; routeFactory.ts:175-176 (IMMUTABLE_FIELDS)	F10 · IDOR Em conversation_notes, authorUserId é imutável mas assignedUserId é mutável no PUT; um agent dono da conversa pode reatribuir a nota a outro usuário.
Alta	domain/whatsapp.ts (WAHA_DEV_PLACEHOLDER_KEY); wahaWebhook.ts:41-54 (PLACEHOLDER_SECRETS)	F11 · Chaves expostas O placeholder 'dev_plaintext_change_me' está hardcodado no código-fonte. readWahaConfig/usableSecret o tratam como 'não configurado' (fail-closed), mas a constante viva no source é um segredo-valor conhecido e commitado.
Média	.dev.vars.example:31-45; wahaWebhook.ts:86-90	F12 · Chaves expostas WAHA_HMAC_SECRET vazio no exemplo deixa o receber em 503 (fail-closed), mas a presença de WAHA_WEBHOOK_ALLOW_UNSIGNED='true' documentada permite operar sem assinatura em desenvolvimento — padrão de config insegura.
Informativa	waha/docker-compose.waha.yml, waha/.env (fora de src/)	F14 · Chaves expostas Config local do WAHA (compose + .env) não auditada neste escopo; pode conter a chave de API e o segredo HMAC usados pelo engine local.
Baixa	views/**/*.ts (innerHTML em dezenas de pontos, ex.: CrmInboxView.ts:127, OrdersView.ts:46)	F15 · XSS Todo o render usa innerHTML com template string. A proteção depende 100% do escapeText/escapeAtrib aplicado em cada dado dinâmico; não há defesa em profundidade (CSP) nem biblioteca de sanitização dedicada.
Informativa	format.ts:41 (export const escapeHtml = escapeText)	F16 · XSS Alias depreciado escapeHtml exportado (mesma implementação). Não utilizado no código atual; mantém API legada que pode induzir uso incorreto (o nome sugere escape de HTML genérico).

Arquivo:linha aponta a origem primária de cada achado; o corpo do achado completo (evidência, impacto e correção) está na seção 5 (Issues para o GitHub).

4. Recomendações priorizadas

Prioridade	Encontrados	Ação	Resultado esperado
P1	F6	Adicionar coluna/valor de dono a todos os recursos do ERP e estender o mapa OWNERSHIP de routeFactory.ts para ingredients, components, products, customers, orders e stock_movements (ou exigir papel write global para a área ERP e documentar a decisão).	Eliminar escrita/deleção de qualquer ID pelo agent.
P1	F7	Proibir manager de editar qualquer campo de usuário; restringir escrita a 'si mesmo' ou a papéis estritamente inferiores, e exigir admin para deletar (atualização de allowedRoles + checagem de posse na users/[id].ts).	Impedir escrita cruzada e escalada na gestão de equipe.
P2	F8, F9, F10	Incluir quick_replies, tags, appointment_types, pipelines e pipeline_stages em OWNERSHIP (ou torná-los só-admin); validar conversationId contra a posse na criação de mensagem; tornar assignedUserId de conversation_notes imutável.	Fechar o IDOR restante no CRM.
P2	F1, F2	Definir política de escopo de leitura: ou filtrar listagens por ownership/tenant, ou documentar explicitamente que o app é single-tenant com visibilidade total por papel — e estender o recorte do SSE a todos os recursos expostos.	Remover a ambiguidade de isolamento e o vazamento por SSE.
P2	F11, F12, F14	Tirar o valor placeholder do source (carregá-lo só de env), validar em startup que WAHA_API_KEY/WAHA_HMAC_SECRET diferem dos defaults conhecidos e falham fechado; documentar que WAHA_WEBHOOK_ALLOW_UNSIGNED só pode ser true em dev.	Zerar segredo-valor conhecido no código e barrar config insegura no boot.
P3	F4, F5, F15, F16	Validar canal/telefone antes de criar conversa; restringir customerId do novo pedido ao contato da conversa aberta; adicionar CSP + considerar um passo de sanitização explícito; remover o alias escapeHtml.	Profundizar a defesa (em profundidade) contra XSS e fechar os gaps de regra de negócio.

5. Issues para o GitHub

Para cada achado acionável, o texto completo de uma issue em Markdown, pronto para copiar/colar, delimitado por --- ISSUE n --- e --- FIM ISSUE n ---.

--- ISSUE 1 ---

security, crítica

```
### Problema O mapa de posse (`OWNERSHIP`) em `src/server/routeFactory.ts` só lista as tabelas do CRM. As tabelas do ERP (`ingredients`, `components`, `products`, `customers`, `orders`, `stock_movements`) e recursos de apoio (`settings`, `pipelines`, `pipeline_stages`, `tags`, ...) não têm entrada nesse mapa, e o `checkOwnership` devolve `null` (permite) quando a tabela não está listada. ### Por que é explorável Qualquer usuário com papel `agent` (o mínimo para `PUT`/`DELETE`) pode alterar ou excluir qualquer registro do ERP – bastando conhecer o `id`. Não há checagem de posse nem restrição por papel acima do mínimo. ### Evidência `src/server/routeFactory.ts:73-100` ``ts const OWNERSHIP: Record<string, OwnershipMap> = { contacts: { kind: 'column', field: 'assignedUserId' }, // ... apenas tabelas do CRM }; // linha ~100: const om = OWNERSHIP[table]; if (!om) return null; // tabela sem dono definido, permite `` ### Impacto Um `agent` pode danificar o catálogo (ingredientes/produtos/componentes), alterar pedidos ou dar baixa indevida de estoque – escrita/deleção não autorizada em dados críticos do ERP. ### Sugestão de correção - Adicionar coluna/valor de dono às tabelas do ERP e incluir as tabelas em `OWNERSHIP`; ou - Documentar que a área ERP é de escrita global por papel e elevar o mínimo de escrita/exclusão a `manager`/`admin` para essas tabelas. ### Critérios de aceite - [ ] `PUT/DELETE` em `/api/ingredients/:id`, `/api/products/:id`, `/api/customers/:id`, `/api/orders/:id` e `/api/stock-movements` recusam (403) agente não proprietário; - [ ] Teste de regressão prova que agent não-alvo recebe 403 e proprietário/manager/admin passam; - [ ] Matriz de papéis documentada para a área ERP.
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

security, crítica

```
### Problema `PUT /api/users/:id` valida o papel do chamador via `allowedRoles(body)`, mas não verifica a posse sobre o usuário alvo. Um `manager` consegue editar `name`/`email` de qualquer usuário (inclusive de outros managers), e a exclusão fica aberta a quem tem papel de gestão. ### Por que é explorável Há hierarquia de papel (viewer/agent/manager/admin), mas a escrita sobre contas de terceiros não é limitada por posse nem por relação de senioridade. ### Evidência `src/pages/api/users/[id].ts:62-65` ``ts function allowedRoles(body: PutBody): Role[] { const adminOnly = ADMIN_ONLY_FIELDS.some(f => body[f] !== undefined); return adminOnly ? ADMINS : USER_MANAGERS; // manager edita qualquer usuário } `` ### Impacto Escrita cruzada sobre contas de outros usuários; degradação de integridade da equipe e base para manipulação de identidade/e-mail. ### Sugestão de correção - Restringir a edição de um usuário a si mesmo ou a papéis estritamente inferiores; - Exigir `admin` para editar `role`/`password` e para exclusão; - Adicionar checagem de posse/senioridade no handler. ### Critérios de aceite - [ ] Manager não consegue editar outro manager; - [ ] Não-admin não consegue mudar `role` de ninguém; - [ ] Testes cobrem os pares manager→manager e agent→qualquer um (esperado 403).
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

security, alta

Problema Três recortes relacionados deixam escrita não autorizada no CRM: 1. `quick\_replies`, `tags`, `appointment\_types`, `pipelines` e `pipeline\_stages` não estão em `OWNERSHIP` – qualquer `agent` edita/deleta recursos compartilhados. 2. Na criação de `messages`, o `conversationId` vem do body e não é validado contra a posse da conversa. 3. `conversation\_notes`: `assignedUserId` é mutável no `PUT`, permitindo reatribuir a nota entre usuários. ### Evidência - `src/pages/api/crm/quick-replies/[id].ts` (usa `createItemRoutes` sem posse) - `src/server/routeFactory.ts:248-255` (criação deriva dono só quando há coluna direta; `messages` é `via` e não válida na criação) - `src/server/tables.ts:216-224` (CONVERSATION_NOTES_SHAPE) ### Impacto Manipulação do funil (etiquetas/estágios/respostas), injeção de mensagem em conversa alheia e reatribuição indevida de notas. ### Sugestão de correção - Incluir as tabelas de apoio em `OWNERSHIP` (ou torná-las somente-admin); - Validar `conversationId` contra a posse na criação de mensagem; - Tornar `assignedUserId` de `conversation\_notes` imutável. ### Critérios de aceite - [] Agent não dono não edita tags/respostas/estágios de outro usuário; - [] POST de mensagem em conversa alheia retorna 403; - [] PUT em nota não altera `assignedUserId`.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

security, alta

Problema Todos os endpoints de listagem devolvem a base completa, filtrando apenas pelo papel mínimo (viewer). Não há recorte por `assignedUserId`/tenant. O recorte por posse existe só no CRUD individual (CRM) e no SSE para conversas/mensagens. ### Por que é explorável O app é single-tenant por design, mas a ausência de qualquer isolamento de leitura significa que, se a premissa de "um inquilino" mudar (ou houver vazamento de credencial de um agente), todos os dados de todos os usuários ficam expostos. ### Evidência `src/server/routeFactory.ts:224-234` (`listHandler` devolve `listEntities` sem filtro) aplicado em `src/pages/api/\*\*/index.ts`. ### Impacto Leitura em massa de contatos, conversas, mensagens e pedidos por qualquer sessão válida. ### Sugestão de correção - Documentar explicitamente o modelo single-tenant na arquitetura; e/ou - Adicionar filtro opcional por ownership/tenant nos listers (ao menos para dados pessoais de terceiros) e estender o escopo do SSE a todos os recursos expostos. ### Critérios de aceite - [] Decisão de escopo de leitura registrada (single-tenant vs por dono); - [] Se por dono: listagens e SSE filtrados por ownership com teste de regressão.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

security, alta

Problema O placeholder `dev\_plaintext\_change\_me` está hardcodado em `src/domain/whatsapp.ts` e `src/server/wahaWebhook.ts`. A leitura trata o valor como "não configurado" (fail-closed), mas o segredo-valor conhecido vive commitado no código. O `dev.vars.example` ainda documenta `WAHA\_WEBHOOK\_ALLOW\_UNSIGNED=true` como escape, permitindo operar sem assinatura. ### Por que é explorável Se um ambiente não sobrescrever a chave e a validação de startup for relaxada, o default vira segredo real e previsível. ### Evidência `src/server/wahaWebhook.ts:41-54` (`PLACEHOLDER\_SECRETS`), `src/domain/whatsapp.ts` (`WAHA\_DEV\_PLACEHOLDER\_KEY`), `dev.vars.example:31-45`. ### Impacto Chave previsível de API/HMAC em um cenário de configuração incompleta. ### Sugestão de correção - Carregar o valor default apenas de env, removendo a constante do source; - Validar no boot que `WAHA\_API\_KEY`/`WAHA\_HMAC\_SECRET` diferem dos defaults conhecidos e falham fechado; - Restringir `WAHA\_WEBHOOK\_ALLOW\_UNSIGNED` a ambientes de dev (nunca em produção). ### Critérios de aceite - [] Nenhum literal de segredo conhecido no source; - [] Boot falha (503/abort) quando a chave é default/ausente; - [] Teste prova que `allowUnsigned` não é aceito em produção.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

security, média

```
### Problema - O compositor de pedido da inbox associa o cliente do contato em aberto, mas `POST /api/orders` aceita qualquer `customerId` do body. - A UI só abre conversa para contatos com telefone, porém `POST /api/crm/conversations` valida apenas o papel. ### Evidência `src/ui/views/crm/CrmInboxView.ts:492-506`; `src/pages/api/orders/index.ts`; `src/ui/views/crm/CrmContatosView.ts:205-211`. ### Impacto Escrita associada a recurso fora do contexto (pedido para cliente alheio; conversa sem canal viável). ### Sugestão de correção - Validar o `customerId` contra o contato da conversa (ou exigir posse do cliente) no handler do pedido; - Validar canal/telefone na criação de conversa. ### Critérios de aceite - [ ] Pedido com `customerId` fora da conversa aberta → 403/422; - [ ] Conversa sem canal/telefone válido → 422.
```

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

security, média

```
### Problema Todo o render usa `innerHTML` com template string; a proteção depende exclusivamente de `escapeText`/`escapeAtrib` em cada dado dinâmico. Não há CSP nem sanitização em segunda etapa, e o alias depreciado `escapeHtml` continua exportado. ### Evidência `src/domain/format.ts:33-47`; `src/ui/views/crm/CrmInboxView.ts:127`; dezenas de usos de `innerHTML`. ### Impacto Um único ponto de escape esquecido vira XSS de armazenamento; não há barreira de contenção. ### Sugestão de correção - Adicionar CSP (csp header / meta) para contenção de XSS; - Considerar uma etapa de sanitização explícita em conteúdo longo (ex.: notas/mensagens); - Remover o alias `escapeHtml` e manter apenas `escapeText`/`escapeAtrib`. ### Critérios de aceite - [ ] CSP presente e testado; - [ ] `escapeHtml` removido; - [ ] Teste de regressão de XSS por vetor (nome, mensagem, nota, tag).
```

--- FIM ISSUE 7 ---